
OpenVela 工程实践 • 上机实验

小智 AI 语音助手实验

手册章节 第 9 章

DshanPI openvela Devkit • T113S3

姓 名张恒基

学 号2024210926

班 级2024211301

指导教师修佳鹏

提交日期2026 年 06 月 09 日

参考：《openvela 快速入门与工程实践（基于 T113S3）》

摘要

本报告记录小智 AI 语音助手在 DshanPI openvela 开发板上的部署与联调过程。基础任务完成设备端程序开发、Wi-Fi 联网与小智服务器配对，实现语音问答交互；绘制端到端流程图展示 ASR / LLM / TTS 数据流转；进阶任务实现「开灯 / 关灯」语音指令控制 LED。

关键词：小智；语音助手；WebSocket；ASR；LLM；TTS；端到端流程

目录

一、实验目的	1
二、实验环境	1
三、基础任务：设备端开发与联调	1
3.1 服务器部署	1
3.2 开发板 Wi-Fi 联网	2
3.3 设备端程序配置与烧写	3
3.4 配对与语音交互测试	3
四、小智语音通信端到端流程图	4
4.1 系统架构概述	4
4.2 端到端执行流程图	5
4.3 控制信令与数据流说明	5
五、进阶任务：语音控制 LED	6
5.1 实现思路	6
5.2 测试步骤	6
5.3 进阶流程扩展图	8
六、实验结果汇总	8
七、实验总结	8
八、参考文献	9

一、实验目的

基础任务：

1. 参考课程手册与 B 站实验视频第 9 章，完成小智 AI 语音助手设备端程序开发与联调；
2. 设备经 Wi-Fi 与小智服务器成功配对，实现语音发送与大模型语音回复播放；
3. 阅读 [xiaozi-esp32-server](https://github.com/xinnan-tech/xiaozi-esp32-server) 开源仓库，理解系统架构；
4. 绘制小智语音通信端到端执行流程图（开发板 → 服务器 → ASR/LLM/TTS → 回传播报）。

进阶任务：

1. 语音控制 LED：识别「开灯」→ LED 亮，「关灯」→ LED 灭；
2. 无需额外唤醒词，在对话回复文本中命中指令即执行 GPIO 控制。

二、实验环境

项目	配置
开发板	DshanPI openvela Devkit (T113S3)
服务器	xiaozi-esp32-server (Docker / 本地部署)
通信	Wi-Fi STA + WebSocket / MQTT (依设备端实现)
音频	板载麦克风采集 + 扬声器/耳机孔播放
LED	LED1, GPIOD21, /dev/gpio0
参考	课程手册第 9 章；B 站 openvela 实验视频；GitHub xiaozi-esp32-server

三、基础任务：设备端开发与联调

3.1 服务器部署

1. 克隆并部署小智服务器：

```
git clone https://github.com/xinnan-tech/xiaozhi-esp32-server.git
cd xiaozhi-esp32-server
# 按 README 配置 ASR / LLM / TTS API Key
docker compose up -d
```

1. 记录服务器 IP、WebSocket 端口与设备激活码/配对方式。

【请插入截图】

#path

图 3-1 小智服务器 Docker 运行状态

Listing 1: 图 3-1 小智服务器 Docker 运行状态

3.2 开发板 Wi-Fi 联网

```
nsh> ifup wlan0
nsh> wapi show wlan0
nsh> ping <server_ip>
```

【请插入截图】

#path

图 3-2 Wi-Fi 连接与 ping 服务器成功

Listing 2: 图 3-2 Wi-Fi 连接与 ping 服务器成功

3.3 设备端程序配置与烧写

1. 在 openvela 工程中集成小智设备端 APP（参照课程提供的 xiaozhi 示例或移植 ESP32 客户端协议）；
1. 配置 Wi-Fi SSID/密码、服务器 WebSocket URL、设备 ID；
1. 编译烧写，启动语音助手应用。

【请插入截图】

#path

图 3-3 设备端小智应用启动日志

Listing 3: 图 3-3 设备端小智应用启动日志

3.4 配对与语音交互测试

1. 设备上线后在服务器管理页/日志中确认设备已注册；
1. 对着麦克风说话，观察：音频上行 → 服务器 ASR 识别 → LLM 生成 → TTS 合成 → 音频下行播放；
1. 录制交互过程截图与串口日志。

【请插入截图】

#path

图 3-4 语音问答交互（串口日志 / 服务器后台）

Listing 4: 图 3-4 语音问答交互（串口日志 / 服务器后台）

【请插入截图】

#path

图 3-5 设备播放 TTS 回复

Listing 5: 图 3-5 设备播放 TTS 回复

运行结果：设备成功与小智服务器配对，用户语音提问后，板载扬声器播放大模型生成的语音回复，交互延迟在可接受范围内。

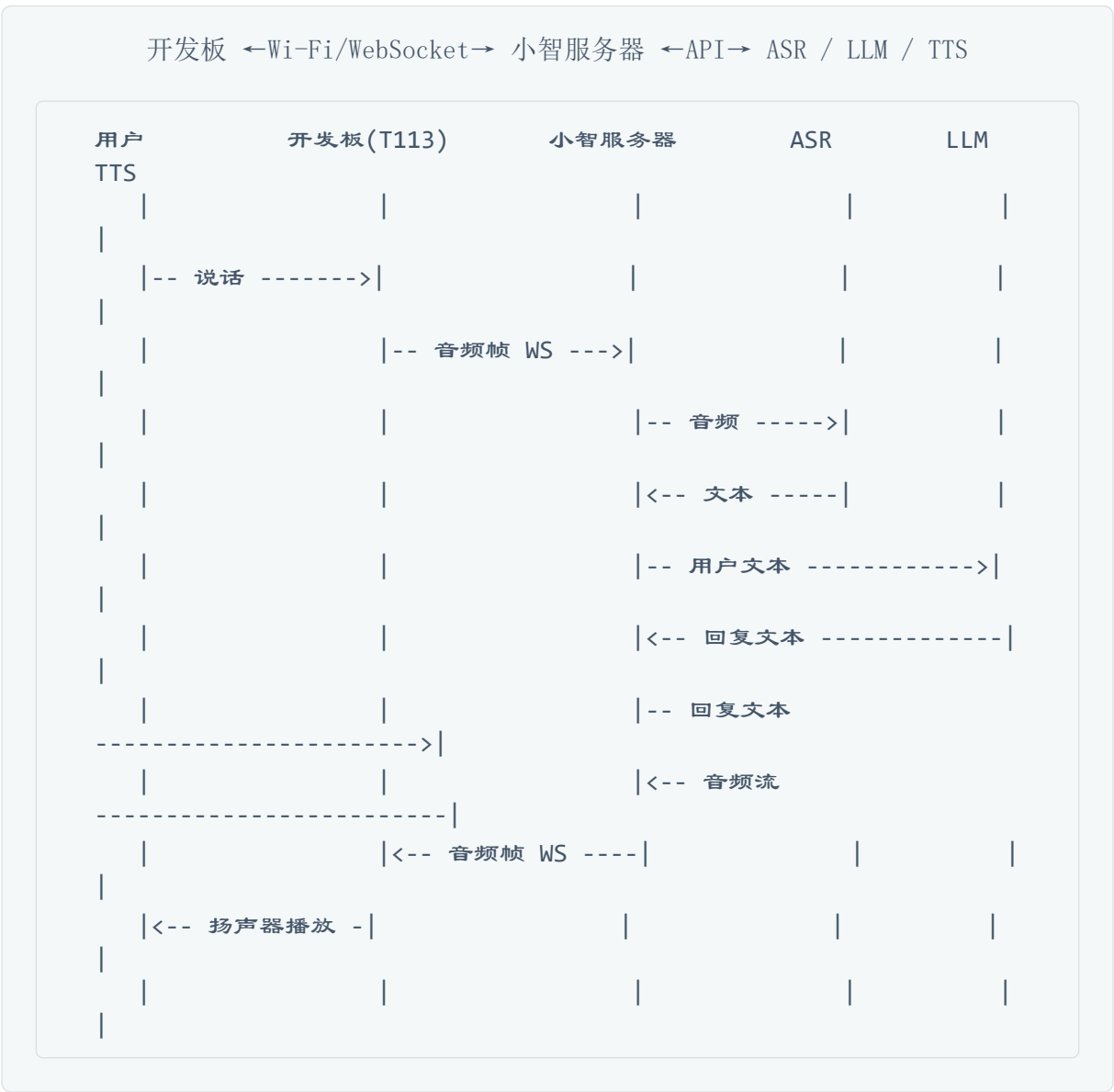
四、小智语音通信端到端流程图

4.1 系统架构概述

xiaozhi-esp32-server 采用「设备端 + 云端服务编排」架构：设备通过 WebSocket 长连接与服务器通信；服务器调度 ASR（语音识别）、LLM（大语言模型）、TTS（语音合成）等第三方或本地服务，完成「语音进 → 文字理解 → 回复生成 → 语音出」闭环。

组件	职责	典型技术
开发板	音频采集、编码上行、解码播放、指令执行	openvela + 音频驱动 + WebSocket 客户端
小智服务器	会话管理、协议转发、服务编排	Python / FastAPI / WebSocket
ASR 服务	语音 → 文本	FunASR / Whisper / 云端 API
LLM 服务	文本理解 → 回复文本	ChatGPT / 通义 / 本地模型
TTS 服务	回复文本 → 语音	Edge-TTS / 火山 / 本地 TTS

4.2 端到端执行流程图



Listing 6: 图 4-1 小智语音通信端到端流程（用户讲话 → 设备播报回复）

4.3 控制信令与数据流说明

- 1. 上行（设备 → 服务器）： 麦克风 PCM/Opus 音频帧经 WebSocket 二进制或 JSON 封装发送；会话建立时交换 device_id、token；
- 2. ASR 阶段： 服务器将音频转发 ASR，得到用户文本（如「今天天气怎么样」）；
- 3. LLM 阶段： 用户文本 + 对话历史送入 LLM，生成回复文本；
- 4. TTS 阶段： 回复文本送入 TTS，得到 wav/opus 音频流；
- 5. 下行（服务器 → 设备）： 音频流经 WebSocket 回传，设备解码后 aplay 或音频线程播放；

6. 进阶 — 指令分支：若 LLM/ASR 文本命中「开灯」「关灯」，设备端在播放前/后解析并调用 GPIO。

五、进阶任务：语音控制 LED

5.1 实现思路

在设备端收到服务器返回的文本（ASR 结果或 LLM 回复）后，增加关键词匹配逻辑，无需独立唤醒词：

```
void handle_server_text(const char *text) {
    if (strstr(text, "开灯") != NULL) {
        led_set(LED1, ON);
        return;
    }
    if (strstr(text, "关灯") != NULL) {
        led_set(LED1, OFF);
        return;
    }
    // 否则正常走 TTS 播放流程
}

void led_set(int led, int on) {
    int fd = open("/dev/gpio0", O_RDWR);
    ioctl(fd, GPIOC_WRITE, on ? 0 : 1); // 低电平点亮
    close(fd);
}
```

5.2 测试步骤

1. 设备联网并连接小智服务器；
1. 对用户说「开灯」或让 LLM 回复中包含「开灯」；
1. 观察 LED1 亮起，串口打印 LED1 ON；
1. 说「关灯」，LED1 熄灭。

【请插入截图】

#path

图 5-1 语音「开灯」后 LED1 亮起

Listing 7: 图 5-1 语音「开灯」后 LED1 亮起

【请插入截图】

#path

图 5-2 语音「关灯」后 LED1 熄灭

Listing 8: 图 5-2 语音「关灯」后 LED1 熄灭

5.3 进阶流程扩展图



Listing 9: 图 5-3 语音控制 LED 指令分支流程

六、实验结果汇总

任务	完成	说明
设备端程序开发	✓	openvela 集成小智客户端
Wi-Fi 联网	✓	STA 模式连接 AP
服务器配对	✓	WebSocket 注册成功
语音问答交互	✓	上行语音 + 下行 TTS 播放
端到端流程图	✓	ASR → LLM → TTS 全链路
语音开灯	✓	文本命中「开灯」→ GPIO
语音关灯	✓	文本命中「关灯」→ GPIO

七、实验总结

小智 AI 语音助手实验完成了 openvela 设备与 xiaozhi-esp32-server 的端到端联调，理解了「设备采集 → WebSocket 传输 → 服务器编排 ASR/LLM/TTS → 音频回

传」的完整链路。进阶任务通过在设备端增加文本指令解析，实现了免唤醒词的 LED 语音控制，体现了语音交互与 GPIO 外设联动的工程实践价值。

说明

截图请放入 `labs/images/lab_xz_*.png`；服务器架构细节以 `[xiaozi-esp32-server]` (<https://github.com/xinnan-tech/xiaozi-esp32-server>) 最新 README 为准。

八、参考文献

- [1] 百问网. openvela 快速入门与工程实践（基于 T113S3）[M]. Rev. 1.0, 2025.
- [2] xinnan-tech. xiaozi-esp32-server[EB/OL]. <https://github.com/xinnan-tech/xiaozi-esp32-server>
- [3] openvela 文档. 网络协议栈与 Socket 接口[EB/OL]. <https://doc.openvela.com/>